Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Here's a comprehensive detail on **Amazon DynamoDB**, covering its core features, architecture, real-world use cases, and a curated set of **interview questions with answers** to help you master it:

What Is Amazon DynamoDB?

Amazon DynamoDB is a fully managed, serverless NoSQL database service offered by AWS. It supports both **key-value** and **document** data models with seamless **high performance**, **horizontal scalability**, and **enterprise-grade reliability**—including multi-AZ and multi-region support.

(AWS Documentation)

- **Schema flexibility**: It's schemaless beyond the primary key, so your data can evolve over time.
- **Read/write performance**: Designed for single-digit millisecond latency, even at massive scales.
  (AWS Documentation)
- **Fully managed & serverless**: No infrastructure or scaling hassles for developers.
  (AWS Documentation)

Key Features & Capabilities

Scalability & Availability

- **Multi-AZ replication** ensures high availability and durability.
- **Global Tables** enable automatic multi-region, multi-active replication with ~99.999% uptime.
  (AWS Documentation)

Data Consistency & Transactions

- Supports **ACID transactions** across items and tables for robust business logic.
  (AWS Documentation)

Indexing & Queries

- **Global Secondary Indexes (GSI)** allow querying on non-primary key attributes and can be modified at runtime.
- **Local Secondary Indexes (LSI)** share the partition key but have a different sort key; must be defined at table creation.
  (Interview Baba, AWS Documentation)

Performance & Throughput Modes

- Choose between:
  - **Provisioned capacity** (with auto-scaling) – you manage RCUs and WCUs.
  - **On-demand capacity** – pay-per-request, great for unpredictable workloads.
    (Second Talent, dynobase.dev)
- For read-heavy apps needing microsecond latency, use **DynamoDB Accelerator (DAX)**.
  (Second Talent, InterviewPrep)

Event Streams & Backup

- **DynamoDB Streams** capture change data in real-time—great for event-driven architectures.
  (AWS Documentation)
- **Point-in-Time Recovery (PITR)** keeps continuous backups for up to 35 days.
- **On-demand backups** for archiving or compliance.
  (Amazon Web Services, Inc., Second Talent, InterviewPrep)

Security & Integrations

- Provides **encryption at rest** via AWS KMS and fine-grained access control using IAM and attribute-based policies.
- Supports **VPC endpoint (PrivateLink)** for secure access.
  (Amazon Web Services, Inc.)
- Integrates with Lambda, API Gateway, AppSync, Redshift, S3, and more—enabling serverless and analytics-driven workflows.
  (AWS Documentation)

TTL & Auto Scaling

- Define **Time-to-Live (TTL)** on items for automatic deletion and storage cost reduction.
- Built-in auto-scaling supports dynamic throughput adjustment for cost optimization.

(AWS Documentation, dynobase.dev)

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Interview Questions & Answers

1. What are primary key, partition key, and sort key?

A **partition key** uniquely identifies items and determines data placement. A **composite primary key** includes both a partition key and sort key, enabling structured querying and organization.

(hellointern.in, Wikipedia)

2. Difference between GSI and LSI?

- **GSI**: Flexible, can be added later, uses different partition and sort keys.
- **LSI**: Defined at table creation, shares partition key with table but has a different sort key.

  (Interview Baba, mindmajix)

3. Consistency models in DynamoDB?

- **Eventually consistent reads** (default) offer lower latency and cost.
- **Strongly consistent reads** guarantee latest data at the cost of higher latency.

  (Interview Baba)

4. What is DAX and when to use it?

DAX is an in-memory read cache for DynamoDB delivering microsecond latency. Best for highly read-intensive workloads, with minimal code changes.

(Second Talent, InterviewPrep)

5. Explain DynamoDB Streams.

A real-time change log capturing data modifications. Can trigger Lambda functions for custom workflows.

(AWS Documentation, dynobase.dev)

6. Provisioned vs On-demand capacity

- **Provisioned**: User-defined capacity and costs.
- **On-demand**: Automatically scales with usage; ideal for unpredictable workloads.

  (dynobase.dev, Second Talent)

7. How does DynamoDB ensure durability and recovery?

- **Multi-AZ replication** ensures resilience.
- Partition data across multiple leader/electable nodes using Paxos.
- **AutoAdmin** handles partition scaling and recovery.
- **PITR** supports restoration to any point in the past 35 days.

  (Wikipedia, InterviewPrep, dynobase.dev)

8. What is ACID transaction support in DynamoDB?

Supports atomic, consistent, isolated, durable transactions across multiple items. Useful for payment processing or inventory management. (AWS Documentation)

9. When would you NOT use DynamoDB?

Use RDBMS instead when:

- Relationships and Joins are complex.
- Schema queries are ad-hoc or unknown ahead of time.
- Complex aggregations are needed.

(Reddit)

10. How to optimize large attribute reads and cost?

Project only necessary attributes using indexes to reduce capacity usage and cost.

(Reddit)

Summary Table

Feature	DynamoDB Highlights
NoSQL Model	Key-value & document; schemaless
Scalability	Unlimited, highly available across AZs & regions
Indexing	Support for GSI & LSI
Performance Optimization	DAX (microsecond cache), Streams, auto-scaling
Durability	Multi-AZ replication, PITR, encryption, backup & restore capabilities

Master AWS DynamoDB: 10 essential interview questions with answers on AWS DynamoDB!

Owner: DGR Uploads

https://www.youtube.com/watch?v=L9kg1SCLCx0&t=1s

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Amazon DynamoDB setup, use cases

Let's go **deep** into **Amazon DynamoDB** setup and its **real-world use cases** with examples, architecture diagrams, and best practices. I'll break this into sections so it's easy to follow:

1. Amazon DynamoDB Setup on AWS

DynamoDB is **serverless** and **fully managed**, so the setup is straightforward. However, designing it **properly** for performance, scalability, and cost optimization is critical.

Step 1 — Create a DynamoDB Table

Steps in AWS Console:

1. Go to **AWS Console → DynamoDB**.
2. Click **Create Table**.
3. Provide:
    - **Table Name** → e.g., Orders
    - **Primary Key**:
        - **Partition Key**: OrderID (String)
        - Optional **Sort Key**: CustomerID
4. Select **Provisioned** or **On-Demand Capacity**:
    - **Provisioned** → You define RCU/WCU.
    - **On-Demand** → AWS auto-scales based on requests.
5. Enable **Point-in-Time Recovery** (PITR) for continuous backups.
6. Choose **Encryption** (default KMS or custom key).
7. Click **Create Table**.

Step 2 — Enable Indexes for Flexible Queries

- **Global Secondary Index (GSI):**
    - Use when you want to query **non-primary key attributes**.
    - Example: Query Orders by CustomerID.
- **Local Secondary Index (LSI):**
    - Use when you want a **different sort key** on the same partition key.

Step 3 — Integrate DynamoDB Streams

- **DynamoDB Streams** capture **real-time changes**.
- Can be integrated with:
  - **AWS Lambda** → Event-driven processing.
  - **Amazon Kinesis** → Real-time analytics.
  - **Amazon S3** → Archiving changes.

Step 4 — Enable DynamoDB Accelerator (DAX)

- Use **DAX** when you have **read-heavy workloads**.
- Provides **microsecond latency** using in-memory caching.

Step 5 — Secure & Optimize

- Use **IAM Policies** for fine-grained access control.
- Enable **VPC Endpoints** for private access.
- Configure **Auto Scaling** to optimize costs.
- Use **TTL** (Time-to-Live) to delete old data automatically.

Step 6 — Connect DynamoDB with Applications

Ways to Connect:

- **AWS SDKs** → Java, Python (Boto3), Node.js, Go, etc.
- **API Gateway + Lambda + DynamoDB** → For **serverless applications**.
- **AppSync + DynamoDB** → For GraphQL APIs.

2. DynamoDB Use Cases

Amazon DynamoDB is widely used where **scalability**, **low latency**, and **high availability** are critical. Below are **8 key use cases** with examples:

2.1. E-commerce Platforms 🛍️

Scenario: Storing and querying **product catalogs**, **customer profiles**, and **order histories**.

- **Why DynamoDB?**
  - Handles **millions of concurrent users**.
  - Auto-scales during traffic spikes.
  - Integrates with **Lambda** for real-time order processing.

Example Architecture:

- **Frontend:** React / Angular
- **Backend:** API Gateway + Lambda
- **Database:** DynamoDB (Orders, Customers, Products)
- **Caching:** DAX
- **Analytics:** Redshift + Kinesis

2.2. Gaming Leaderboards 🎮

Scenario: Maintaining **real-time scores** and **rankings** for millions of players.

- **Why DynamoDB?**
  - Sub-millisecond writes for **live score updates**.
  - **LSIs** to query by PlayerID and timestamp.
  - **Streams** trigger Lambda to update global leaderboards.

2.3. IoT Device Data Storage 🌐

Scenario: Storing billions of **sensor readings** from IoT devices.

- **Why DynamoDB?**
  - Handles **high write throughput**.
  - Works seamlessly with **Kinesis** for streaming analytics.
  - TTL automatically deletes outdated sensor data.

2.4. Real-Time Messaging & Chat Apps 💬

Scenario: Storing conversations and delivering instant messages.

- **Why DynamoDB?**
  - Stores **billions of chat messages** efficiently.
  - Integrates with **AppSync** for GraphQL-based subscriptions.
  - Streams push new messages in **real-time**.

2.5. Financial Transactions 💳

Scenario: Payment processing and fraud detection.

- **Why DynamoDB?**
  - **ACID transactions** ensure data integrity.
  - Real-time fraud detection using **Lambda + DynamoDB Streams**.
  - Multi-region replication for global availability.

2.6. Ticket Booking Systems 🎟️

Scenario: Handling millions of concurrent seat bookings.

- **Why DynamoDB?**
  - Uses **conditional writes** to prevent double-booking.
  - High throughput during peak traffic (e.g., concerts, flights).
  - TTL auto-expires unused reservations.

2.7. Healthcare Records

Scenario: Securely storing **patient data** and **medical histories**.

- **Why DynamoDB?**
  - HIPAA-compliant.
  - Attribute-based IAM policies for **data-level security**.
  - Integrates with **S3** for document storage.

2.8. Machine Learning Data Storage

Scenario: Storing feature sets and model results for ML pipelines.

- **Why DynamoDB?**
  - Stores **real-time inference results**.
  - Integrates with **SageMaker**, **Athena**, and **Glue**.
  - Supports event-driven retraining workflows.

3. DynamoDB Reference Architectures

3.1. Serverless Web Application

React Frontend → API Gateway → AWS Lambda → DynamoDB → DAX

↓

DynamoDB Streams → Kinesis → Redshift/S3

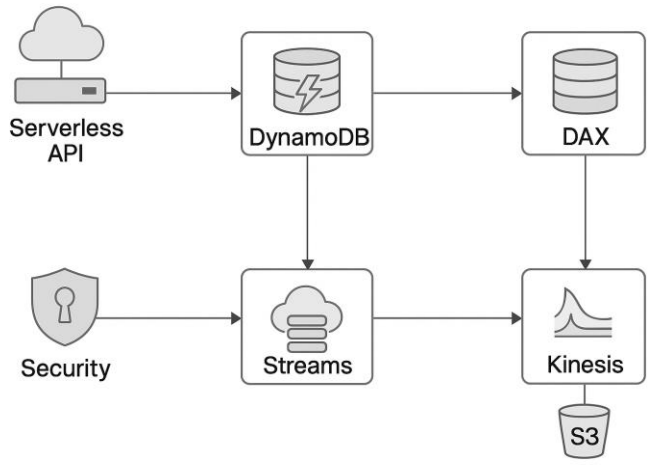
3.2. Global Multi-Region Setup

User → Route53 → API Gateway → Lambda → DynamoDB Global Tables → Multi-Region Replication

4. Best Practices

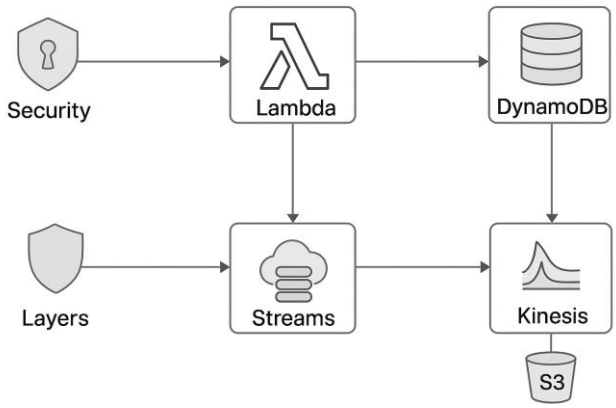
Area	Best Practice
Capacity	Use **on-demand** for unpredictable traffic; **provisioned** for steady workloads
Indexes	Use **GSIs** for cross-attribute queries; **LSIs** for range queries
Performance	Use **DAX** for read-heavy apps
Cost Optimization	Enable **TTL**, use **auto-scaling**, and project fewer attributes
Security	Use **KMS encryption** and fine-grained IAM roles
Analytics	Stream data to **Kinesis** or **Redshift** for reporting
Resilience	Use **Global Tables** for active-active multi-region setups

AWS DynamoDB architecture diagram showing serverless API integration, DAX caching, Streams, Kinesis analytics, S3 backups, and security layers:

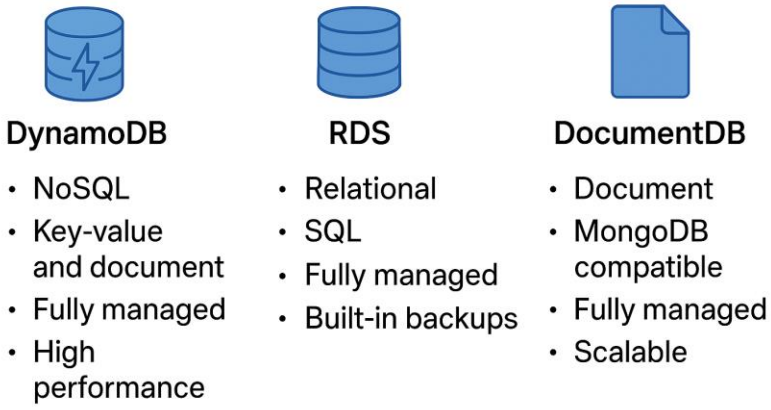


A high-quality AWS architecture diagram showing:

DynamoDB
API Gateway
Lambda
DAX
Streams → Kinesis → S3
Security Layers



A comparison diagram: DynamoDB vs RDS vs DocumentDB.



Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Amazon DynamoDB — Hands-On Lab Guide

1. Lab Prerequisites

Before starting, ensure you have:

- An **AWS account** with admin privileges
- **AWS CLI v2** installed
- **Python 3.x** with **Boto3** SDK
- **PowerShell 7+** (optional)
- **IAM user** with AmazonDynamoDBFullAccess
- **AWS Region:** us-east-1 (recommended for lab)

2. Creating Your First DynamoDB Table

2.1 Using AWS Console

1. Go to **AWS Console → DynamoDB → Tables → Create Table**.
2. Set:
    - Table name: **Orders**
    - Partition key: OrderID *(String)*
    - Sort key: CustomerID *(String)*
3. Choose **On-Demand Capacity** *(best for unpredictable workloads)*.
4. Enable:
    - **Encryption**: AWS KMS (default)
    - **PITR**: ✅
5. Create the table.

2.2 Using AWS CLI

aws dynamodb create-table \

```
  --table-name Orders \
  --attribute-definitions \
    AttributeName=OrderID,AttributeType=S \
    AttributeName=CustomerID,AttributeType=S \
  --key-schema \
    AttributeName=OrderID,KeyType=HASH \
    AttributeName=CustomerID,KeyType=RANGE \
  --billing-mode PAY_PER_REQUEST
```

2.3 Using Python Boto3

```
import boto3
dynamodb = boto3.resource('dynamodb')
table = dynamodb.create_table(
  TableName='Orders',
  KeySchema=[
    {'AttributeName': 'OrderID', 'KeyType': 'HASH'},
    {'AttributeName': 'CustomerID', 'KeyType': 'RANGE'}
  ],
  AttributeDefinitions=[
    {'AttributeName': 'OrderID', 'AttributeType': 'S'},
    {'AttributeName': 'CustomerID', 'AttributeType': 'S'}
  ],
  BillingMode='PAY_PER_REQUEST'
)
print("Table Status:", table.table_status)
```


3. Loading Sample Data

3.1 AWS CLI

```
aws dynamodb put-item \
  --table-name Orders \
  --item '{
    "OrderID": {"S": "1001"},
    "CustomerID": {"S": "CUST001"},
    "Amount": {"N": "250"},
```

```
    "Status": {"S": "Pending"}
  }'
```

3.2 Python SDK

```
table = dynamodb.Table('Orders')
table.put_item(
  Item={
    'OrderID': '1001',
    'CustomerID': 'CUST001',

    'Amount': 250,
    'Status': 'Pending'
  }
)
```

---

4. Creating Secondary Indexes

4.1 Global Secondary Index (GSI)

Query orders by **Status** instead of OrderID.

```
aws dynamodb update-table \
  --table-name Orders \
  --attribute-definitions AttributeName=Status,AttributeType=S \
  --global-secondary-index-updates \

"[{\"Create\":{\"IndexName\":\"StatusIndex\",\"KeySchema\":[{\"AttributeName\":\"Status\",\"KeyType\":\"HASH\"}],\"Projection\":{\"ProjectionType\":\"ALL\"},\"ProvisionedThroughput\":{\"ReadCapacityUnits\":5,\"WriteCapacityUnits\":5}}}]"
```

---

5. Enabling DynamoDB Streams

Streams capture **real-time changes** to your table.

```
aws dynamodb update-table \
  --table-name Orders \
  --stream-specification StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES
```

Integrate **Streams → Lambda** to trigger events when new orders are inserted.

---

6. DynamoDB Accelerator (DAX)

For **microsecond latency** in read-heavy apps:

```
aws dax create-cluster \
  --cluster-name OrdersDAX \
  --node-type dax.r5.large \
  --replication-factor 3 \
  --iam-role-arn arn:aws:iam::<ACCOUNT_ID>:role/DAXRole
```

---

7. Enabling TTL for Expiry

Automatically delete **old records**:

```
aws dynamodb update-time-to-live \
  --table-name Orders \
  --time-to-live-specification "Enabled=true, AttributeName=ExpiryDate"
```

---

8. Securing DynamoDB

Security Layer	Implementation
IAM	Fine-grained access policies per table
KMS Encryption	Default encryption for data at rest
TLS (HTTPS)	Enforce encryption in transit
VPC Endpoints	Private connectivity via **Interface Endpoints**
Secrets Manager	Store API keys, IAM creds securely
CloudWatch	Monitor throttles, errors, latency
CloudTrail	Log all API calls

---

9. DynamoDB + Lambda + API Gateway Architecture

```
React App → API Gateway → AWS Lambda → DynamoDB Table → DAX Cache
                                  |
                                  └── DynamoDB Streams → Kinesis → Redshift/S3
```

---

10. Real-Time Use Cases

Use Case	Why DynamoDB
E-commerce Orders	Handles millions of concurrent requests

Use Case	Why DynamoDB
Gaming Leaderboards	Sub-ms latency for player scores
IoT Data Storage	Billions of writes/sec
Messaging Apps	Real-time chat sync
Financial Systems	ACID transactions + high durability

---

11. DynamoDB Interview Questions & Answers

Beginner Level

1. **Q:** What is DynamoDB?
    - **A:** A fully managed NoSQL key-value and document database.
2. **Q:** Difference between Provisioned & On-Demand capacity?
    - **A:** Provisioned = fixed RCU/WCU; On-Demand = auto scales.

Intermediate Level

3. **Q:** When to use GSI vs LSI?
    - **A:** GSI = query across all partitions; LSI = query within a partition.
4. **Q:** How do DynamoDB Streams work?
    - **A:** Capture table changes in real time, consumed by Lambda or Kinesis.

Advanced Level

5. **Q:** How do DynamoDB Global Tables work?
    - **A:** Provide **multi-region active-active replication**.
6. **Q:** How do you optimize DynamoDB costs?
    - **A:** TTL, auto-scaling, sparse indexes, projected attributes.

---

12. Next Steps

I can also prepare:

- A **high-quality AWS architecture diagram** showing:
    - DynamoDB
    - API Gateway
    - Lambda
    - DAX
    - Streams → Kinesis → S3
    - Security Layers
- A **comparison chart**: **DynamoDB vs RDS vs DocumentDB**.

Do you want me to create a **professional AWS DynamoDB architecture diagram** showing **serverless API integration, DAX caching, Streams, Kinesis analytics, S3 backups, and security layers**? It'll be a high-quality reference for interviews and projects. Should I?